

INTERNET

- interconnects billions of computing devices throughout the world.
- use the TCP/IP protocol as the foundation for communication between devices.
- connects diverse networks. so any two hosts can communicate.

NUTS AND BOLTS VIEW

1. END DEVICES

- devices running network apps
e.g. TVs, gaming console, home security systems etc.

2. PACKET SWITCHES

- takes a packet from incoming communication link and forwards it to outgoing communication links.
- many shapes and flavors.
 - ↳ routers, link-layer switch.
- at the destination host, they are reassembled into the original data.
- packet from source host to destination host follows a route/path.

3. COMMUNICATION LINKS

- end systems are connected together through this.
e.g. coaxial cable, copper wire, optical fibre, radio.

4. TRANSMISSION RATE

- different links transmit data at different rates.
- measured in bits/second.

5. PROTOCOLS

- controls sending and receiving of information within the internet.
 - ↳ Transmission Control Protocol (TCP)
 - ↳ Internet Protocol (IP): specifies format of packets being sent.
- Internet standards.
 - ↳ RFCs: documents defining protocols & practices.

Date: _____

SERVICES VIEW

- distributed applications since they involve multiple end systems.
- applications run on end systems, not on packet switch or network core.
- "socket interface" specifies how a program running on one end system asks the Internet infrastructure to deliver data to a specific destination program running on another end system.
- set of rules that must be followed.

PROTOCOLS

1. HUMAN PROTOCOLS

- specific messages are sent and specific actions we take in response.

2. NETWORK PROTOCOLS

- entities exchanging messages are hardware or software components.
- all communication is governed by a protocol.
- "protocols define the format, order of messages sent and received between two entities, and actions taken on message transmission or receipt"

NETWORK EDGE

1. NETWORK EDGE

- end systems as they sit at the edge of internet; also known as hosts.
- hosts are divided into clients and servers
- clients are user devices e.g. desktop, laptop.
- servers are powerful machines that store and distribute web pages, stream video.

Q. ACCESS NETWORKS

→ physically connects an end system to the first router

→ Home Access: Digital Subscriber Line

- it is accessed from the same local telephone company that provides wired access.
- DSL modem connects to DSLAM located in telco local central office.
- it takes digital data and translates it to high-frequency tones for transmission over telephone wires to the central office.
- residential telephone lines carry both data and traditional telephone signals simultaneously; encoded at different frequencies.
- on the customer side, a splitter separates data and telephone signals arriving to the home and forwards to DSL modem.
- thousands of households connect to a single DSLAM.

→ Home Access: Cable (HFC)

- use of cable television company's existing cable television infrastructure.
- fibre optics connect the cable head end to neighbourhood-level junctions, traditional coaxial cable is then used to reach individual houses.
- cable internet access requires cable modems.
- cable modems connects to cable modem termination system.
- it turns the analog signal sent from the cable modem in many downstream homes back into digital format.
- it is a shared broadcast medium.
- if several users are simultaneously down-loading a file on the downstream channel, the actual rate will be lower as that of the aggregate cable.

Date: _____

→ Home Access: Fiber to the Home (FTTH)

- provides an optical fiber path from the central office directly to the home.
- It can provide access rates in gigabits per second.
- 2 optical-distributions network architecture that splits fiber into individual customer specific needs when close to home are ADNs and PONs.
- Active optical networks (ADNs) is ethernet based.
- It requires active equipment, has high performance and higher cost.
- Passive optical networks (PONs) uses optical splitters to combine number of homes onto a single shared optical fiber which connects to DLT in the CO. It is connected to the Internet via telco router. At home users connect a home router to the ONT

→ Home Access: 5G Fixed Wireless

- provides high speed residential access
- uses beam-forming technology; data is sent wirelessly from a providers base station to the modem in a house.

→ Access in Enterprise (and Home): Ethernet and WIFI

- local area network is used to connect an end system to the edge router.
- ethernet is the most prevalent LAN technology in universities and homes.
- ethernet users use twisted-pair copper wire to connect to an ethernet switch. It is then connected to a larger Internet.
- wireless users receive packets to/from an access point that is connected into the enterprise's network, which is connected to wired internet.

→ Wide-Area Wireless Access: 3G and LTE 4G and 5G

- devices employ the same wireless infrastructure used for cellular telephone to send/receive packets through a base station that is operated by the cellular network provider.
- provides mobile broadband connectivity over large geographical areas using cellular networks.
- users only need to be within few tens of kilometers.

→ Physical Media

- bits travel from source to destination through a series of transmitter-receiver pairs. For each pair the bit is sent by propagating EM waves or optical pulse across a physical medium.

• Guided Media

- ★ waves are guided along a solid medium such as:
 - ★ Fibre-optic cable: for long distance & high speed (10s or 100s gigabits/sec)
 - ★ Twisted copper: less expensive, ethernet, two insulated wires in spiral.
 - ★ coaxial cable: high data transmission rates, cable TV, 2 concentric wires

• Unguided Media

- ★ waves propagate in atmosphere and outer space. such as wireless LAN

- ★ radio wave, microwave, infrared.

- ★ Terrestrial Radio channel: carry signals in EM spectrum, requires no physical wire to install, can penetrate walls, for WIFI and cellular; path loss & shadow fading as signals travel over a distance and around obstructing objects, multipath fading and interference as reflections cause signal fluctuation and affected by other devices/networks.

Terrestrial Radio Channels.

• bluetooth, WIFI, cellular



Date:

★ Satellite Radio channels: a communication satellite links two or more Earth based microwave transmitter/receiver known as ground stations; Geostationary satellites have fixed spot on Earth, huge distance but causes signal propagation delays; Low-earth orbiting satellites are much closer to earth and no fixed spot, to provide continuous coverage, many satellites need to be placed in orbit.

★ have global coverage but deployment cost is heavy.

3. NETWORK CORE

→ mesh of packet switches & links that interconnects the internet's end system.

NETWORK CORE

1. SWITCHING / FORWARDING

- transfer data from sender to receiver across a network through a path.
- because of multiple paths switching decides which path to take.
- scalable communication.
- there are 2 types, (a) packet switching (b) circuit switching.

(a) PACKET SWITCHING

- the source message breaks long messages into smaller chunks of data known as packets.
- each packet travels through communication links and packet switches
- routers and link layers are 2 types of packet switches.
- Transmission rate of a link = R bits/sec

$$\text{Transmission time / delay for one packet} = \frac{L}{R} = \frac{\text{packet size (bits)}}{\text{transmission rate (bits/sec)}}$$

$$\text{Packets} = \frac{\text{message size (M)}}{\text{packet length}}$$

$$\text{Total Delay} = \frac{M}{R}$$

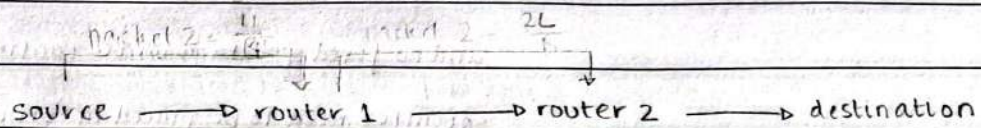
Date:

→ Store and Forward Transmission

- packet switch must receive the entire packet before transmitting first bit of packet onto the outbound link.

Total end to end transmission delay = $N \frac{L}{R}$

N = no. of hops.
 L = packet size
 R = link rate



• at time $\frac{L}{R}$
 packet 1 = router 1 → router 2

packet 2 = source → router 1
 packet 3 = source

• at time $2 \frac{L}{R}$
 packet 1 = router 2 → dest
 packet 2 = router 1 → router 2
 packet 3 = source → router 1

• at time $3 \frac{L}{R}$
 packet 1 = already at dest
 packet 2 = router 2 → dest
 packet 3 = router 1 → router 2

• at time $4 \frac{L}{R}$
 packet 3 = router 2 → dest

→ **Queuing**

- each packet switch has multiple links attached to it.
- for each link there is an output buffer, it stores packets that the router is about to send into that link.
- if a link is busy, the arriving packet waits in a queue, causing queuing delay.
- buffer space is finite and this can cause packet loss if buffer is completely full with other packets waiting. (arrival rate > transmission rate)

→ **Packet forwarding and routing**

- source includes destination's IP address in the packet's header
- each router has a forwarding table that maps destination addresses to routers outbound links.
- routing protocols are used to set forwarding tables, paths taken by packets.

(b) CIRCUIT SWITCHING

→ resources needed along a path (buffer, transmission rate) are reserved for the duration of the communication session between end systems.

→ commonly used in traditional telephone, requires a connection to be established.

$$\text{Dedicated Rate} = \frac{\text{Transmission Link Rate}}{\text{Number of Circuits}}$$

→ **Frequency Division Multiplexing**

- frequency spectrum of a link is divided up among the connections established across the links, a frequency band is dedicated to each connection.

→ **Time Division Multiplexing**

- time is divided into frames of fixed duration, each frame is divided into fixed time slots.

(c) PACKET SWITCHING VS CIRCUIT SWITCHING

- packet switching offers better sharing of transmission capacity
- it is simple & more efficient

2. NETWORK OF NETWORK

- host connects to internet via an access ISP which must be interconnected.
- if each access ISP directly connects to every other access ISP, it would be costly. $\rightarrow O(n^2)$
- the solution is to interconnect all of the access ISP with a single global transit ISP.
- access ISP is a customer and global transit ISP is said to be the provider
- the 2nd network structure is if every company builds their global transit ISP, it will compete with original global transit; thousands of access ISPs and multiple global transit ISP
- it is a 2 tier hierarchy with global transit providers residing at top tier
- regional ISP to which the access ISPs in the region connect $\rightarrow$ it then connects to tier 1 ISP.

PERFORMANCE: LOSS, DELAY, THROUGHPUT

- as a packet travels from source to destination, a packet suffers several types of delays; nodal processing, queuing, transmission, propagation delay.

1. NODAL PROCESSING DELAY

- time required to examine the packets header and determine where to direct the packet
- time taken to check for bit-level errors
- $< \text{microseconds}$

Date: _____

2. QUEUING DELAY

- At the queue, it experiences delay as it waits to be transmitted onto the link.
- length of queuing delay depends on the number of packets already in the queue waiting for transmission onto the link.
- if queue is empty, then queuing delay is 0.

3. TRANSMISSION DELAY

- packets are transmitted on first come-first-served basis, so all the packets arrived before should be transmitted first.
- microseconds - milliseconds.

$$d_{\text{TRANS}} = \frac{L}{R} = \frac{\text{packet length}}{\text{transmission rate}}$$

- time required to push all of the packets bits onto the link.

4. PROPAGATION DELAY

- once a bit is pushed, it needs to propagate to router B.
- propagation speeds depends on the physical medium of the link. (2×10^8 m/sec - 3×10^8 m/sec)

$$d_{\text{PROP}} = \frac{d}{s} = \frac{\text{distance btw 2 routers}}{\text{propagation speed of link}}$$

- milliseconds.

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

QUEUING DELAY

- a : average packet arrival rate. (packets/sec)
- R : transmission rate (bits/sec)
- L : packet length (bits)

$$\text{Traffic Intensity} = \frac{La}{R}$$

* $La/R > 1$: average queuing delay is large/infinite

$La/R \leq 1$: " " " is large

$La/R \approx 0$: " " " is small.

Date: _____

PACKET LOSS

- a queue preceding a link has finite capacity.
- a packet arriving to a full queue, the router will drop that packet hence packet is lost.
- fraction of lost packages increases as traffic intensity increases.

THROUGHPUT

- rate at which bits are being sent from A to B.
- instantaneous : any instant time
- average : rate over T seconds

$$\text{average throughput} = F/T = \text{bits/sec}$$

- for simple two-link network, throughput is $\min(R_c, R_s)$, it is the transmission rate of bottleneck link.
- if $R_s < R_c \rightarrow$ throughput is R_s
- if $R_c < R_s \rightarrow$ throughput is R_c

PACKET LOSS

- a queue preceding a link has finite capacity.
- a packet arriving to a full queue, the router will drop that packet hence packet is lost.
- fraction of lost packages increases as traffic intensity increases.

THROUGHPUT

- rate at which bits are being sent from A to B.
- instantaneous : any instant time
- average : rate over T seconds

$$\text{average throughput} = F/T = \text{bits/sec}$$

- for simple two-link network, throughput is $\min(R_c, R_s)$, it is the transmission rate of bottleneck link.
- if $R_s < R_c \rightarrow$ throughput is R_s
- if $R_c < R_s \rightarrow$ throughput is R_c

SECURITY

- bad guys can put malware into your host
 - deleting files and installing spyware that collects our private information.
 - network of thousands of similarly compromised devices, known as botnet
 - it is self replicating
- bad guys can sniff packets
 - a passive receiver that records a copy of every packet that flies by is called a packet sniffer.
 - has all sensitive information.
- bad guys can use fake identity
 - ability to inject packets into the Internet with fake source address is known as IP spoofing.

Date:

220 server → TCP connection is given accessed

250 server → successful completion

354 server → receiver received message

221 server → closing the connection (SMTP)

header → sender email address, receiver email address.

→ HTTP takes SMTP to the server.

→ Domain name system; through IP address it communicates, it is unique.

DNS Queries

↳ Recursive

↳ Iterative

→ bad guys can attack servers

- DOS attack renders a network, host, or other piece of infrastructure unusable by legitimate users.

- web servers, email servers can all be subject to DOS.

- can be vulnerability attack, bandwidth flooding, connection flooding.

- large number of packets released to target host, preventing legitimate packets from reaching the server.

→ lines of defense

- authentication

- confidentiality

- integrity checks

- access restrictions, password protected VPN

- firewalls.

PROTOCOLS LAYER, SERVICE MODELS

1. LAYERED ARCHITECTURE

- provides modularity, making changes in implementation easy.
- remainder of system remains unchanged when a layer's implementation is changed.

(a) PROTOCOL LAYERING

• application → transport → network → link → physical

(i) APPLICATION

- network application and protocol such as HTTP, SMTP reside
- HTTP for web document request & transfer
- SMTP for transfer of email
- FTP for transfer of files.
- exchange packet of information.

(ii) TRANSPORT

- transports application layer messages between application endpoints.
- TCP and UDP are transport protocols.
- TCP provides a connection oriented service to its application, also breaks long message into shorter ones.
- UDP provides connectionless service to its applications.

(iii) NETWORK

- moves network layer packets from one host to another.
- transport protocol passes transport layer segment and dest address to the network layer

Date:

(iv) LINK

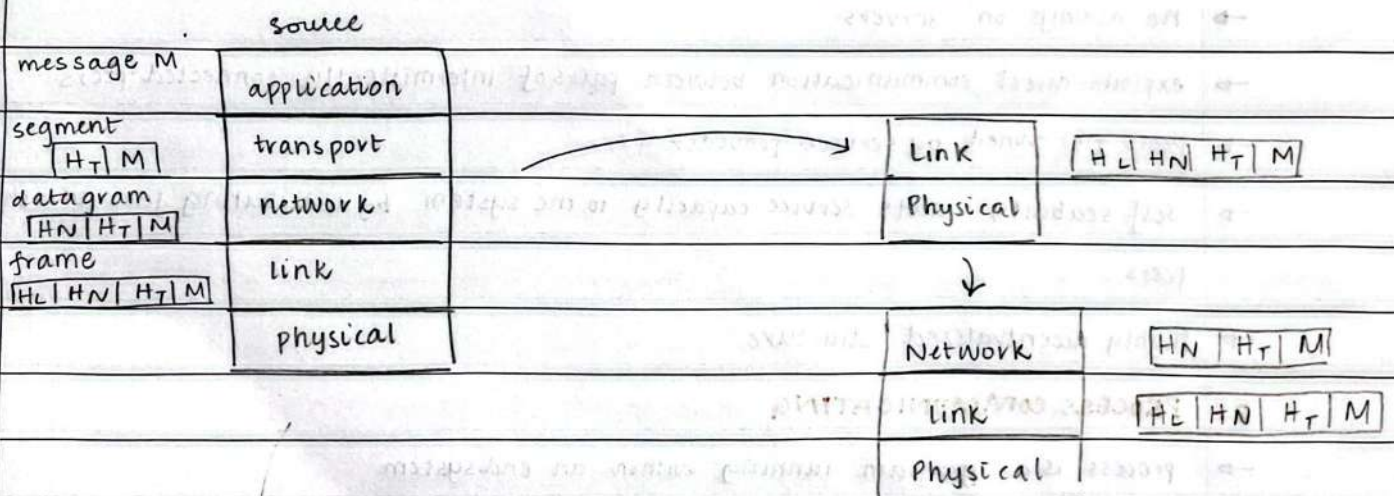
- to move a packet from one node to the next node in the route, the network layer relies on the services of link layer.

(v) PHYSICAL

- moves individual bits within the frame from one node to the next.
- protocols are link dependent.

2. ENCAPSULATION

→ application layer → transport layer → appends header information → transport layer encapsulates application layer message → network layer → appends header info → datagram passed to link layer → appends header info → link layer frame



destination
same as source ←

PRINCIPLES OF NETWORK APP

1. ARCHITECTURES**(a) CLIENT - SERVER**

- server : always on host
- client : services request from many other host.
- clients do not communicate directly with each other.
- server has a fixed, IP address.
- client can always contact the server by sending packet
- a data center is used to create a powerful virtual server having large no. of hosts.
- e.g. HTTP, FTP.

(b) PEER TO PEER

- no always on servers.
- exploits direct communication between pairs of intermittently connected peers.
- peers not owned by service provider
- self scalability : adds service capacity to the system by distributing files to other peers.
- highly decentralized structure

2. PROCESS COMMUNICATING

- process is a program running within an end system.
- within same host, IPC takes place.
- on different hosts, they communicate with exchanging messages.

Date:

→ sockets

- a process sends messages into, receives messages from, the network through a software is called socket
- without socket, server wouldn't know that the request is meant for the web app.
- it is the interface between application layer and transport layer.

→ to receive message you must have IP address and port number.

→ many processes can run on same host so IP address of dest must be known.

→ HTTP port is 80

3. TRANSPORT SERVICES. AVAILABLE TO APPLICATIONS.

→ reliable data transfer

- cannot afford to lose data.

→ throughput

- some apps require minimum amount of throughput to be effective
- elastic applications can use as much throughput as it is available.

→ timing

- for non real-time applications, lower delay is always preferable to higher delay, but no tight constraint on end to end delay.

→ security

- confidentiality, data integrity & end point authentication.

4. TRANSPORT SERVICES PROVIDED BY THE INTERNET

→ TCP Service

- connection oriented : setup required between client & server.
- reliable data transfer: all data sent without error and in order.
- congestion control: throttles sending process when network is congested.
- does not provide min. throughput guarantee, security.

→ UDP service

- connectionless, so no handshaking
- unreliable data transfer, no guarantee that message will reach the receiving process.
- messages may arrive out of order.
- does not include congestion control mechanism.

5. SECURING TCP

- neither TCP nor UDP provides encryption.
- Transport Layer Security provides critical process to process security service, integrity, encryption.

6. APPLICATION LAYER PROTOCOL

- types of messages exchanged ; request or response
- syntax of messages
- meaning of info in fields.
- rules for when & how process sends message & responds.
- some application layer protocols are specified in RFC, are in public domain.
- many other are proprietary, not available to public domain e.g. skype.

Date:

WEB & HTTP

→ a web page consists of objects which is simply a file such as HTML, JPEG.

→ most web pages consist of a base HTML file & several referenced objects.

→ client: browsers that request & displays web objects.

→ server: sends objects in response

→ HTTP uses TCP

- it initiates TCP connection with the server.

- browser & server processes access TCP through their socket interfaces.

- HTTP server & client send and receive message from socket.

- TCP provides reliable data service.

- HTTP server maintains no information about the clients, it is said to be a stateless protocol.

→ HTTP connection types

- non-persistent

- ★ TCP connection initiated on port 80

- ★ HTTP request to server via socket.

- ★ HTTP server receives request message via socket.

- ★ TCP connection closed.

- ★ at most one object sent over TCP

- ★ Round Trip Time: time it takes for a small packet to travel from client to server & back to client.

- 1 RTT to initiate connection, 1 RTT for HTTP request & response to return

response time = 2RTT + file transmission

Date:

- persistent

- ★ server leaves TCP connection open after sending a response.
- ★ subsequent request between same client & server can be sent over same connection.
- ★ When server receives back to back requests, it sends object back to back.
- ★ as little as one RTT for all referenced objects.

→ HTTP message format

- request

- ★ first line is the request line.
- ★ subsequent lines are header.

method field - URL field - HTTP version field

↓
GET, POST, HEAD, PUT, DELETE

- ★ GET: when browser requests an object, when object is identified in URL

entity body is empty with GET method.

- ★ POST: used to send data to the server, data is sent in entity body.

secure for sensitive data, not cached or stored in history.

used for logins, registrations.

- ★ HEAD: only fetches headers, used for debugging, faster than GET

- ★ PUT: allows user to upload object to a specific path.
entire file must be sent in one request.

- ★ DELETE: allows user to delete an object on web server

Date:

• response

★ first line is status line

★ 6 header lines and then entity body.

◦ Date indicates when HTTP response was created & sent by server.

◦ Content Type : type of text.

★ 200 OK : successful request

★ 301 Moved Permanently: new URL specified in location of response header.

★ 400 Bad Request : request not understood by server.

★ 404 Not Found

★ 505 HTTP Version not supported.

→ Cookies

• allows sites to keep track of users.

• cookie header line in HTTP response message, cookie file kept on users system, managed by users browser, backend database at website.

• without cookies, server would treat every request as a new user

• cookies store user preference, like, theme.

• they store products left in cart

• used to create a user session layer on top of stateless HTTP.

• track user behaviour on a given website (1st party cookie)

• track user behaviour across multiple websites (3rd party)

WEB CACHING

- also called proxy server
- satisfies HTTP requests on behalf of an origin web server.
- has its own disk storage & keeps copies of recently requested objects.
- 1. → browser establishes TCP connection to Web cache & sends HTTP request for the object
- 2. → web cache checks if a copy is saved locally, if it does it returns object with HTTP response to client
- 3. → if it does not, cache requests object from origin server, cache receives object and send it to client
- acts as a server when it receives request from.
- acts as a client when it sends request and receive responses from origin.
- it reduces response time for a client request, reduces traffic on institutions access link.
- Conditional GET allows a cache to verify that its objects are up to date.
- client specifies data of browser cached copy & server responds with no object if up to date.

HTTP/2

- reduce perceived latency by enabling request & response multiplexing over a single TCP connection.
- breaks each message into small frames, schedule frames to mitigate HOL blocking (short frames have to wait behind larger ones)
- HTTP/3 adds more security, error & congestion control

EMAIL

→ 3 major components : user agents, mail servers, Simple Mail Transfer protocol (SMTP)

→ user agents : users to read, reply, forward, save, compose.

e.g. Gmail

→ mail servers: message is saved in outgoing message queue and mailbox for receivers.

→ SMTP: uses TCP to transfer mail from the sender's mail server to the recipient's mail server.

SMTP

→ does not use intermediate mail server

1 → client SMTP has TCP establish to port 25

2 → server & client perform application layer handshaking

3 → SMTP client indicates the email ID of sender & the email ID of recipient

4 → After introducing, client sends message.

5 → SMTP relies on TCP to get the correct message across server.

6 → TCP closes connection, if no other messages are to be sent.

→ SMTP uses persistent connections.

→ has ASCII command/response interaction.

→ requires message to be in 7-bit ASCII

→ message ends in CR: carriage return LF: line feed.

→ SMTP is client push protocol

→ IMAP used to retrieve from server, or deletes or move messages whereas HTTP is used ~~to send~~ as an interface and is using client pull protocol.

DOMAIN NAME SYSTEM

- identifiers include hostname & IP address
- DNS is a distributed database implemented in a hierarchy of DNS servers, and an application layer protocol that allows host to query distributed database
- DNS employed by HTTP and SMTP to translate hostname → IP address.
- desired IP address is often cached in a nearby DNS server, which helps to reduce DNS network traffic.
- other services.

- host aliasing: a host can have one or more alias names;

the actual host name is canonical.

- mail server aliasing:

- load balancing: amongst replicated servers, for replicated web server,

a set of IP address is associated with one alias

hostname

- in centralized design, clients simply direct all queries to the single DNS server, and DNS server responds directly to the querying clients.

- problems of centralized design:

- single point of failure: DNS server crashes, so does Internet.

- Traffic volume: single DNS would have to handle all DNS queries.

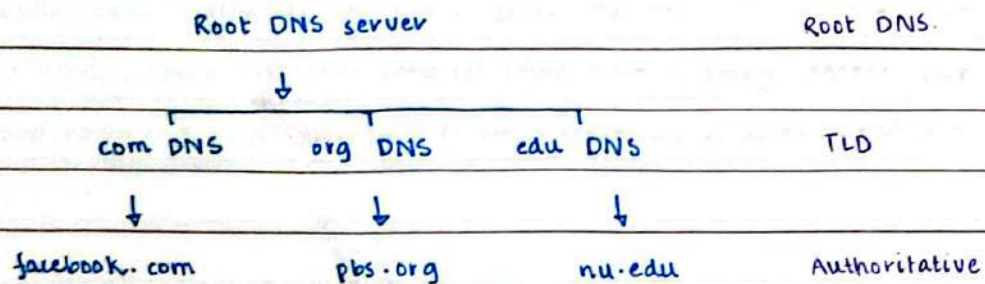
- Distant centralized database: single DNS cannot be closer to all querying clients.

- maintenance: would have to keep records for all Internet hosts.

Date:

→ a distributed, hierarchical database

- client first contacts one of the root servers, which returns IP address for TLD servers
- client then contacts one of these TLD servers, which returns IP address for authoritative server.
- client contacts authoritative server that returns IP address for host name



- authoritative DNS server can be maintained by organization that maps names of publicly accessed hosts to IP address

→ types of DNS queries

- recursive: resolver is responsible for finding the final IP address.
- iterative: resolver may return a referral to another DNS server

DNS CACHING

- improves delay performance & reduce number of DNS messages
- DNS servers discard cached information after a period of time.
- browser → OS → ISP DNS → Global DNS
- local DNS server can also cache the IP address of TLD servers, allowing the local DNS server to bypass the root DNS server in a query chain.
- when a DNS server receives DNS reply, it can cache the mapping in its local memory.
- caching improves response time

Date:

DNS RECORDS & MESSAGES

- DNS stores resource records (RRs)
- each DNS replies carry one or more records.

(Name, Value, Type, TTL)

Type A ; name is hostname, value is IP address

Type NS ; name is domain, value is hostname of authoritative server.

Type CNAME ; name is alias name for some canonical name, value is canonical name.

Type MX ; value is canonical name of mail server that has alias hostname name

PEER TO PEER FILE DISTRIBUTION

- minimal or no reliance on 'always-on' infrastructure servers.
- pairs of intermittently connected hosts, communicate directly.
- PC's, laptops controlled by users.
- each peer can redistribute any portion of the file it has received to any other peers.

FILE DISTRIBUTION

- in client-server, none of the peers helps in distributing the file.
 - ↳ server must transmit one copy of the file to each of the N peers
 - ↳ server transmits NF bits.
 - ↳ time to distribute the file NF/u_s
 - ↳ download rate of peer with lowest download rate $d_{\min} = \min[d_1, d_2, \dots, d_n]$
 - ↳ minimum client download time is $F/d_{\min}$ seconds.

$$\begin{array}{l} \text{time to distribute file (F)} \\ \text{to N clients} \end{array} \rightarrow D_{cs} \geq \max \left\{ \frac{NF}{u_s}, \frac{F}{d_{\min}} \right\}$$

- ↳ distribution time increases linearly with the peers, N .

- in P2P, each peer can assist the server in distributing the file.
- a peer can use its own upload capacity to redistribute the data to other peers.
- since in the beginning only the server has the file, so it must send each bit of the file at least once into its access link so min. distribution time is F/u_s and now the peers may redistribute the bit among themselves.
- minimum distribution time is $F/d_{\min}$.
- total upload capacity is equal to the upload rate of server + individual peers so minimum distribution time becomes $\frac{NF}{u_s + \sum u_i}$

$$\begin{array}{l} \text{time to distribute F} \\ \text{to N clients} \end{array} \rightarrow D_{P2P} \geq \max \left\{ \frac{F}{u_s}, \frac{F}{d_{\min}}, \frac{NF}{u_s + \sum u_i} \right\}$$

Date:

BITTORRENT

- all peers participating in the distribution of a particular file is called a torrent.
- peers in a torrent download equal size chunks of the file from one another.
- peers have no chunks in start, but over time accumulate more and more chunks.
- while it downloads chunks, they also upload it to other peers.
- if a peer has received the entire file, it may leave the torrent or remain in torrent to upload chunks to other peers.
- requesting and sending chunks.
 - ↳ When a new peer joins, they receive IP address of all other peers on torrent through tracker.
 - ↳ some peers leave in between, so they fluctuate over time.
 - ↳ if a peer has L neighbours, then they will obtain L list of chunks, in this way a peer can request for chunks they don't have.
 - ↳ important decision to make are; which chunks to request? And which neighbours to request from?
 - ↳ to decide which chunks, a technique called rarest first is used where the chunks that are rarest amongst the neighbours are to be requested.
 - ↳ to decide from whom to request, give priority to neighbours who are constantly uploading chunks at a very high rate. After every 10 seconds, this is recalculated to check top 4 neighbours. These peers are unchoked.
 - ↳ one additional neighbour after every 30 seconds is optimistically unchoked

Date:

VIDEO STREAMING AND CONTENT DISTRIBUTION NETWORKS

- huge amount of traffic and storage
- important performance measure is average end to end throughput.
- create multiple version of same video, at different quality level
- spatial coding ; many neighbouring pixels in single frame have similar values so send only repeated patterns .
- temporal redundancy : instead of sending entire next frame , send only difference between consecutive frames .
- constant bit rate : video encoding rate fixed
- variable bit rate : " " changes .
- server to client bandwidth will vary over time with congestion levels.
- packet loss may occur , result in poor quality.
- continuous playout constraint
 - ↳ When a client plays a video, playout timing must match original video timing.
 - ↳ network delays can vary so use a client-side buffer - a temporary storage to smooth out variations.
 - ↳ introduces playout delay. — time before playback starts

STREAMING MULTIMEDIA : DASH

- video is encoded into several different versions, each version has a different bit rate and quality .
- client requests chunks of video segments of a few seconds in length.
- if available bandwidth is high, it selects chunks from high rate version, otherwise low.
- allows clients with different internet access rate to stream in video at different encoding rates.

- each video version is stored in HTTP server, each with a different URL.
- HTTP server has a manifest file. that provides URL for each version.
- streaming video = encoding + DASH + playout buffering.

CONTENT DISTRIBUTION NETWORK

- how to stream content to hundreds of thousands of simultaneous users?
 - ↳ build a single massive data center
 - not all clients will be near to the data centre.
 - wastage of network bandwidth if same video is sent again.
 - single point of failure
- a CDN manages servers in multiple geographically distributed locations., stores copies of the video in its server
- can be private or third party
- CDN adopts two different server placements
 - ↳ enter deep : deploying several clusters in access ISP's all over the world - to get closer to end users.
 - ↳ bring home : building large clusters at a smaller number of sites., lower maintenance & management overhead.

Date:

SOCKET PROGRAMMING WITH UDP AND TCP

- client and server communicate with each other by reading from and writing to sockets.
- two socket types are:

TCP and UDP

- TCP: connection oriented, reliable byte-stream channel through which data flows.
- UDP: connectionless and sends independent packets of data. without guarantee of delivery.

SOCKET PROGRAMMING WITH UDP

- When sending a packet, destination address is attached.
- after passing through sender's socket, internet uses destination address to route the packet through the Internet to the socket in the receiving process.
- receiver will extract packets content.

SOCKET PROGRAMMING WITH TCP

- handshake and establish a TCP connection.
- When creating TCP connection, we associate it with client socket address and server socket address.
- just drop data into TCP connection via socket.

TRANSPORT LAYER SERVICES

- provides logical communication between application processes running on different hosts.
- transport layer protocols are implemented in the end-systems but not in network routers.
- transport layer converts application-layer messages into packets known as transport layer segments.
- it passes the segment to the network layer where the segment is encapsulated within a network layer packet & sent to the destination.
- on receiving side, network layer extracts transport layer segment from datagram and passes the segment up to the transport layer.
- TCP and UDP are available to network applications
- TCP
 - ↳ reliable, connection-oriented service
 - ↳ flow control allows sequence number, acknowledgements and timers to be delivered from sending process in order.
 - ↳ converts IP's unreliable service between end systems into reliable data transport.
 - ↳ congestion control - prevents TCP connection from swamping the links and routers between communicating hosts with an excessive amount of traffic.
- UDP
 - ↳ unreliable - does not guarantee that the data sent will arrive intact.
 - ↳ unregulated - can send connection at any rate.

Date:

MULTIPLEXING AND DEMULTIPLEXING

- extending host to host delivery service provided by the network layer to a process-to-process delivery service for application running on host.
- at destination host, transport layer receives segment from network layer.
- transport layer sends data to a socket then to the process, but not directly.
- each socket has a unique identifier because there can be many.
- each transport layer has fields to identify receiving socket and then directs the segment to it.
- demultiplexing: delivering data in transport layer to correct ~~sockets~~ sockets
- multiplexing: gathering data chunks at the source host from different sockets, encapsulating it with header information to create segment, and passing it to network layer
- how demultiplexing works?
 - ↳ each socket in the host could be assigned a port #
 - ↳ transport layer examines the dest port in the segment & directs segment to corresponding socket.
 - ↳ segment data then passes through the socket.

CONNECTIONLESS MULTIPLEXING & DEMULTIPLEXING

- When creating a UDP socket, transport layer assigns a port number not being used by another UDP port.
- UDP socket is fully identified by dest IP address & dest port number.
- if two UDP segments have different source IP, but same destination IP, then both segments will be directed to same dest process via same destination socket.
- TCP socket is identified by: source IP, port #, dest IP, port #.

Date: _____

CONNECTION ORIENTED - TCP

- TCP socket is identified by a four tuple
- When TCP segments arrive from network to host, the host uses all 4 values to direct (demultiplex) segment to appropriate socket.
- two arriving TCP segment with different source IP will be directed to two different socket.
- multiple connection can use same port

CONNECTION LESS TRANSPORT - UDP

- UDP takes message from application process → attaches source & destination → passes segment to network layer → encapsulates transport layer segment into IP datagram → best effort attempt to deliver the segment to receiving host → UDP uses destination port to deliver segment data.
- no handshaking
- reasons to use UDP
 - ↳ no congestion control
 - ↳ no connection establishment
 - ↳ no connection state
 - ↳ small packet header overhead. - UDP has only 8 bytes of overhead.
- uses multimedia.
- lack of congestion control can result in high loss rates.
- segment structure.

source port # dest port #

length checksum

application
data.

→

checksum

↳ used by receiving host to check whether errors have been introduced to the segment

↳ whether bits have been altered.

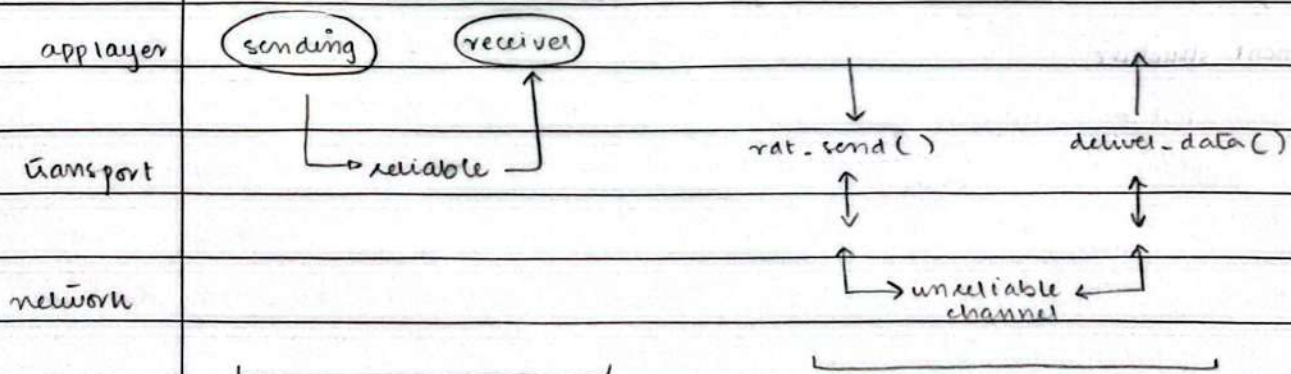
↳ bit errors introduced when a segment is stored in router's memory.

↳ calculated over a few fields in the IP header in addition to UDP segment.

↳ UDP at sender's side performs 1's complement of the sum of 16bit

PRINCIPLES OF RELIABLE DATA TRANSFER

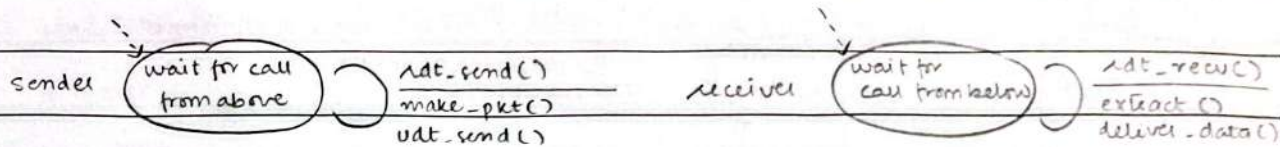
- no transferred data bits are corrupted, or lost, delivered in order.
- layer below reliable data may be unreliable.
- sender and receiver don't know whether a message was received correctly, this can only happen if receiver gives a signal - that's PROTOCOL
- RDT is a protocol to ensure reliable communication over unreliable channels.
- implemented at sender & receiver sides.
- `rdt_send()`: sender uses to send data.
- `rdt_recv()`: receiver uses to receive packet
- `deliver_data()`: receiver delivers data to application layer
- `udt_send()`: sends packet over unreliable channel.
- consider uni-directional data transfer from sender to receiver.



Date:

RELIABLE TRANSFER OVER A RELIABLE CHANNEL : rdt1.0

- underlying channel is completely reliable
- separate finite-state-machine for sender and for receiver.



- sending side simply accepts data from upper layer via `rdt_send()`.
- creates packet and sends it to channel.
- receiving side receives packet from underlying channel via `rdt_rcv()`, removes data from packet and passes data to upper layer.

RELIABLE TRANSFER OVER A CHANNEL WITH BIT ERRORS : rdt2.0

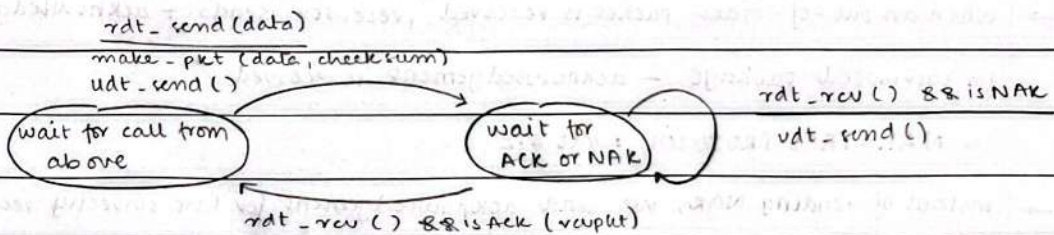
- bits may be corrupted, flipped.
- a bit error occurs when transmitted, propagates or is buffered.
- how to recover from errors?

↳ positive and negative acknowledgements.

↳ sender retransmits packet on negative acknowledgement.

↳ 0 for NAK and 1 for ACK

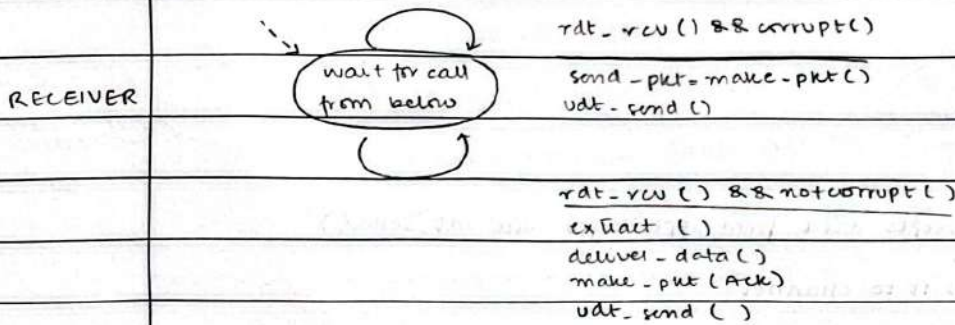
↳ use of checksum



- sender creates a packet and adds checksum, then send packet via `udt_send()`.
- if ACK is received, sender knows recently transmitted packet has been received. → protocol returns to waiting state.

Date:

- if NAK is received then protocol retransmits last packet.
- during ACK or NAK state, it cannot get more data from upper layer.
- protocol known as stop and wait



- receiver has a single state, replies with ACK or NAK
- fatal flaw
 - ↳ possibility that NAK or ACK could be corrupted.
 - ↳ can retransmit but introduces duplicate packets. so receiver doesn't know whether it is new data or just retransmission.

→ solution

- ↳ sender adds sequence number to packets.

SENDER, HANDLING GARBLE NAK/ACK : rdt 2.1

- protocol state for currently sending packet should have a sequence # 0 or 1.
- when an out-of-order packet is received, receiver sends + acknowledgement.
- for corrupted package - acknowledgement is received.

a NAK-FREE PROTOCOL : rdt 2.2

- instead of sending NAK, we send acknowledgement for last correctly received packet.
- sender receives ACK's for same packet twice, knowing that the receiver did not correctly receive the last packet.
- receiver must include sequence # of packet being acknowledged.

Date:

RELIABLE DATA TRANSFER OVER A LOSSY CHANNEL WITH BIT ERRORS : rdt3.0

- underlying channel can lose packets
 - how to detect packet loss? And what to do?
 - ↳ sender transmits a data packet & either packet, or receiver's ACK of that packet gets lost
 - ↳ no reply comes from receiver, so sender waits a reasonable amount of time.
 - ↳ retransmits the data packet.
 - ↳ if packet or ACK is just delayed then retransmission will be duplicate but sequence # handles it.
 - ↳ sender does not know whether a packet was lost, ACK was lost or simply delayed.
 - ↳ requires a countdown timer that can interrupt the sender after a given amount of time.
 - ↳ timer starts when a packet is sent or retransmitted, respond to a timer interrupt and stop the timer.
- $$U_{\text{sender}} = \frac{L/R}{RTT + L/R}$$
- utilization: fraction of time the sender is actually busy sending bits into the channel.
 - stop and wait operation: most of the time its idle
 - sender must be allowed to send multiple packets without waiting for ACKs known as pipelining.
 - ↳ range of sequence # increased as each in-transit packet must have a unique #.
 - ↳ sender/receiver may have to buffer more than one packet; sender to buffer packets that have been transmitted but not ACK.
 - ↳ Go-Back N and selective repeat.

Date: _____

GO-BACK-N: SENDER/RECEIVER

→ no more than N unacknowledged packets in pipeline.

→ must respond to

↳ whether window is full or not to create packets otherwise data is sent back.

↳ cumulative ACK's: ACK's all packets upto seq # n

↳ timer used so if timeout occurs, packet is retransmitted that have been sent but not acknowledged.

SOLVE EXAMPLE

Date:

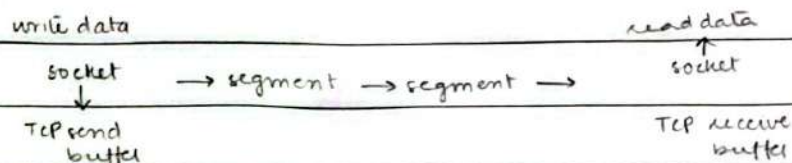
SELECTIVE REPEAT

- avoid unnecessary retransmissions by having the sender retransmit only those packets that it suspects were received in error.
- sender will already receive ACKs for some of the packet.
- out of order packets are buffered until any missing packets are received

SOLVE EXAMPLE

CONNECTION ORIENTED TRANSPORT : TCP

- two processes initiate a handshake
- provides a full duplex service - bidirectional flow
- point to point - single sender and receiver.
- maximum amount of data that can be grabbed and placed in a segment is limited by maximum segment size (MSS).
- TCP pairs each chunk of client data with a TCP header, forming TCP segment
- The segments are passed down to the network layer, where they are separately encapsulated within network layer IP segments/datagrams → IP datagram sent to network → TCP receives a segment at the other end, the segments data is placed in TCP connection receive buffer.



- TCP segment structure includes :
 - ↳ source port # dest port #
 - ↳ sequence number & acknowledgement number.
 - ↳ receive window for flow control
 - ↳ optional and variable length options field.
 - ↳ flag field
- TCP views data as unstructured, but ordered stream of bytes
- sequence number for a segment is byte-stream number of first byte in segment.
- acknowledgement # that host A puts in its segment is the seq # of the next byte Host A is expecting from B.
- for out of order segments, receiver either discards it or keeps the out-of-order bytes and waits for missing byte.

→ Round trip time

↳ timeout should be larger than the connection's RTT - time from a segment is sent until it is acknowledged.

$$\text{Estimated RTT} = (1 - \alpha) * \text{Estimated RTT} + \alpha * \text{SampleRTT}$$

↳ estimated RTT is a weighted average of sampleRTT.

↳ exponential weighted moving average

$$\alpha = 0.125$$

$$\text{Timeout Interval} = \text{Estimated RTT} + 4 * \text{DevRTT}$$

$$\text{DevRTT} = (1 - \beta) * \text{DevRTT} + \beta * |\text{SampleRTT} - \text{EstimatedRTT}|$$

↳ if sampleRTT values have little fluctuation, then DevRTT will be small.

↳ timeout interval shouldn't be too larger than Estimated RTT, otherwise when a segment is lost TCP would not quickly retransmit.

→ TCP sender

↳ uses timeouts to recover from lost segments

↳ TCP receives data from application

↳ each segment includes a seq #.

↳ TCP starts the timer when the segment is passed to IP

↳ expiration interval is TimeoutInterval.

↳ second major event is timeout, it responds by retransmitting the segment that caused the timeout. TCP restarts timer.

↳ 3rd: acknowledgements received, start timer if there are still unacknowledged segment.

→ TCP fast retransmit

↳ retransmitting the missing segment before the segment's timer expires.

→ flow control for TCP

- ↳ eliminate the possibility of the sender overflowing the receiver's buffer.
- ↳ matching the rate at which the sender is sending against the rate at which receiver is reading.
- ↳ sender maintain a variable called 'receive window'
- ↳ receive window tells how much free buffer space is available.
- ↳ $\text{lastbyte} - \text{lastbyte read} \leq \text{rcvbuffer}$ because buffer cannot overflow.

→ TCP connection management

- ↳ establish connection.
- ↳ when 3 packets are sent between the 2 hosts, a three-way handshake initiated.
- ↳ when a connection ends, resources are deallocated.
- ↳ client sends a special TCP segment to the server, this has a flag bit = FIN in header.
- ↳ when server receives this segment, it sends client an ack segment in return.
- ↳ client ack server's shutdown segment.

PRINCIPLE OF CONGESTION CONTROL

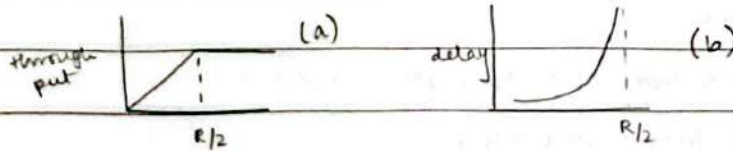
- results from overflowing of router buffers as the network becomes congested.
- too many sources sending data at high rate.

CAUSES / COST OF CONGESTION

→ scenario 1:

- ↳ one router, infinite buffers.
- ↳ allows incoming packet when the packet-arrival rate exceeds the outgoing links capacity.
- ↳ throughput: no. of bytes per second at the receiver.
- ↳ maximum throughput = $R/2$

→ as sending rate exceeds $R/2$, average delay becomes larger.



→ at $R/2$ link is fully utilized in delivering packets.

→ large queuing delays as packet arrival rate nears the link capacity.

→ scenario 2:

↳ a router with finite buffers.

↳ packets will be dropped, if buffer is already full.

↳ sender eventually retransmits

↳ sender times out prematurely & retransmits a packet that has been delayed, but not lost.

↳ un-needed retransmission by the sender in the face of large delays may cause a router to use its link bandwidth to forward un-needed copies of a packet. throughput decreases to $R/4$ as each packet forwarded twice.

→ scenario 3:

↳ 4 senders, router with finite buffer & multi hop paths.

↳ for small values of λ_{in} , buffer overflows are rare, throughput = offered load.

↳ for larger values of λ_{in} , throughput increases.

↳ in high traffic, when a packet is dropped, the work done by first-hop is wasted. The transmission capacity at first router also wasted.

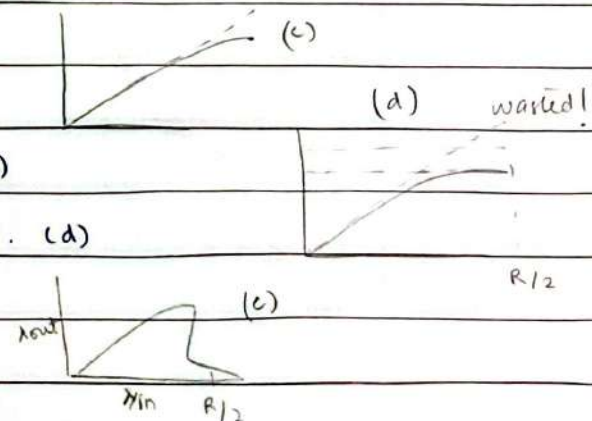
→ throughput can never exceed capacity. (a)

→ delay increases as capacity approached. (b)

→ retransmission decreases effective throughput (c)

→ un-needed duplicates further decreases throughput. (d)

→ transmission capacity wasted for packet lost (e)



APPROACHES TOWARDS CONGESTION CONTROL

→ end to end congestion control

↳ no explicit feedback from network layer → Transport.

↳ congestion inferred from observed loss, delay.

↳ TCP

→ network assisted congestion control

↳ routers provide explicit feedback to sender or receiver.

↳ router informs sender of maximum host sending rate.

TCP CONGESTION CONTROL

→ TCP is to have each sender limit the rate at which it sends traffic into its connection.

→ if there is little congestion, then sender increases its send rate. otherwise decrease.

→ additive increase: increase sending rate by 1 MSS every RTT until loss detected.

→ multiplicative decrease: cut sending rate at half at each loss. on a triple duplicate Ack event.

→ referred to as AIMD → sawtooth - linearly increase its congestion window size until triple duplicate Ack event occurs.

→ TCP sender limits transmission: sender doesn't overwhelm.

↳ $\text{last byte sent} - \text{last byte acked} \leq \min(\text{cwnd}, \text{rwnd})$

TCP rate $\approx \frac{\text{congestion window}}{\text{RTT}}$ bytes/sec

→ congestion window tells how many bytes can be sent before waiting for ACKs.

→ if network is clear, cwnd increases.

→ send cwnd bytes, wait one RTT, send more bytes.

PHASES**1. slow start**

- initially a small value of 1 MSS.
- initial sending rate = MSS/RTT
- the MSS increases by 1 every time a transmitted segment is first acknowledged, hence when first acknowledgement arrives, the congestion window increases by 1 MSS and sends 2 maximum sized segments - so $cwnd += 1$
- grows exponentially.

2. congestion avoidance

- value of $cwnd$ is approximately half its value when congestion was last encountered.
- increases value of $cwnd$ by a single MSS every RTT.
- when $cwnd$ reaches a threshold value ($ssthresh$), TCP switches from slow start to congestion avoidance.

if $cwnd$ starts at 1 MSS :

slow start : $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32$ MSS

CA : $32 \rightarrow 33 \rightarrow 34 \rightarrow 35$

- when packet loss occurs, TCP assumes congestion and cuts $ssthresh$ to $1/2$ of $cwnd$. and process restarts.

3. fast retransmit

- $cwnd$ is increased by 1 MSS for every duplicate ACK received for missing segment.
- if a timeout event occurs then

$cwnd = 1$ MSS

$ssthresh = 1/2 cwnd$

- for duplicate ACK.

$ssthresh = cwnd/2$

$cwnd = ssthresh$.

Date:

→ for 3-duplicate ACKs

↳ TCP Tahoe: $cwnd = 1MSS$ and enters slow phase.

↳ TCP Reno: $cwnd = ssthresh$ and enters fast recovery → congestion avoidance.

→ for timeout

↳ TCP Tahoe: it retransmits after timeout, $cwnd = 1MSS$, enters slow

↳ TCP Reno: " " " " " "

TCP CUBIC

→ congestion window increased on ACK receipt, slow and fast phase remains same.

→ congestion avoidance phase is changed:

↳ W_{max} = congestion window when loss was last detected.

↳ K = future point when TCP cubic window will again reach W_{max} .

↳ increase W as a function of cube of dist. between current time and K .

• larger increase when further from K .

↳ when $t > K$, congestion window increases a small when t close to K .

↳ when congestion loss occurs, CUBIC more quickly ramps up close to W_{max} .

→ Increasing TCP sending rate until packet loss is a bottleneck.

$$\text{average throughput of a connection} = \frac{0.75 * W}{RTT}$$

DELAY BASED CONGESTION

→ measures RTT of source - destination for all acknowledged paths.

Let RTT be the min of these measurements.

$$\text{uncongested throughput rate} = cwnd / RTT$$

→ Increase $cwnd$ linearly if measure throughput close to uncongested.

$$\text{measured throughput} = \frac{\# \text{bytes sent in last RTT}}{RTT_{\text{measured}}}$$

→ maximize throughput while keeping delay low.

EXPLICIT CONGESTION NOTIFICATION

→ at the network layer, two bits in service field of IP datagram header are used for ECN.

→ used to indicate congestion.

→ carried to dest host - which informs sending host.

TCP FAIRNESS

→ if K TCP sessions share same bottleneck link of bandwidth R , each should have average rate of R/K

→ additive increase gives slope of 1 & multiplicative decrease decreases throughput.

→ fairness and UDP

↳ multimedia apps don't use TCP

↳ send video at constant rate & occasionally lose packets.

↔ fairness and TCP

↳ parallel connections between 2 hosts.

↳ decrease transmission rate in face of increasing congestion.

EVOLUTION OF TRANSPORT LAYER FUNCTIONALITY

→ TCP for wireless link, high bandwidth paths with large RTT, packet re-ordering, traffic flow for low priority.

QUIC: QUICK UDP INTERNET CONNECTIONS

→ application layer, on top of UDP.

→ secure HTTP, Google has deployed it.

→ connection oriented & secure

→ allows several different application level streams to be multiplexed through QUIC connection.

→ reliable, TCP friendly.

→ 1 handshake, no HDL blocking.

CHAPTER 4: NETWORK LAYER

OVERVIEW OF NETWORK LAYER

- H1 and H2 are two hosts; network layer in H1 takes segment from the transport layer in H1, encapsulates each segment into a datagram, then sends that to the nearest router.
receiver: at H2, the network layer receives the datagrams from its nearby router, extracts transport layer segments, and delivers it to transport layer at H2.
- router: forward datagrams from input links to output links.
- network control plane: co-ordinate these local, per-router forwarding actions so that datagrams are transferred end to end.

FORWARDING & ROUTING: DATA AND CONTROL PLANES

- 2 Key network layer functions.
 1. forwarding: move packets from input link to the appropriate output link. It takes a few nanoseconds.
Typically used in hardware.
 2. routing: network layer determines route or path taken by packets as they flow from a sender to a receiver.
It is implemented in the control plane through routing algorithms.
- data plane
 - router-local action
 - uses a forwarding table that uses the header values to map the outgoing link interface.
- control plane
 - traditional approach: routing algorithms run in every router and have both forwarding and routing functions to communicate with each other.
 - software-defined networking: uses a remote controller computer and distributes the forwarding tables to be used by each and every router.

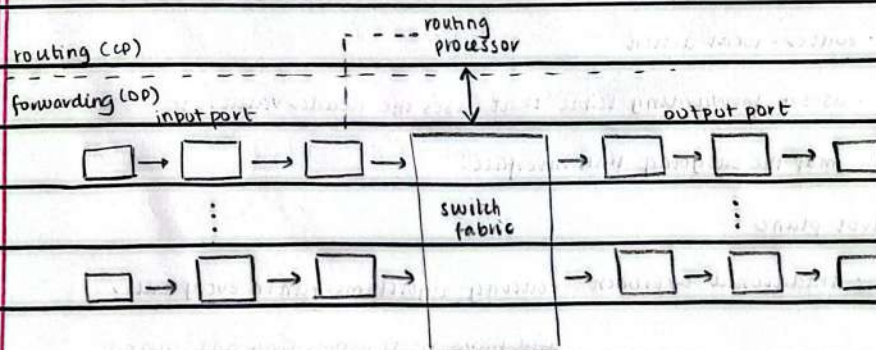
CONTROL PLANE : SDN APPROACH

- separates control plane from data plane
- the remote controller has global view of all routers.
- It computes paths, decides forwarding behaviour, and installs forwarding tables into devices.
- the routers in data plane, they look at packet headers, match against forwarding tables and then forward instead of running routing algorithms.

NETWORK - SERVICE MODEL

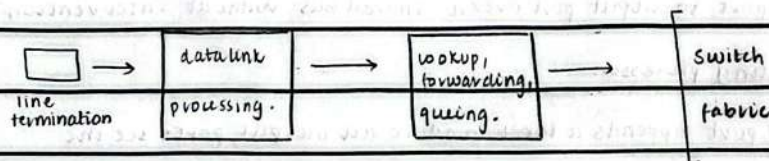
- guaranteed delivery: packet will eventually arrive at destination.
- guaranteed delivery with bounded delay: delivery within a certain time to reach destination.
- in-order packet delivery: packets arrive in order that they were sent.
- guaranteed minimal bandwidth: guarantees a flow at least X Mbps, no matter how busy the network is.
- security: encrypt datagrams at source & decrypt at destination
- other services
 - best-effort: doesn't guarantee delivery, or order of delivery, or minimal bandwidth required.

WHAT'S INSIDE A ROUTER



- input ports : receive packet from incoming links , perform physical and data-link processing , do lookup in the forwarding table to decide output port
- switching fabric : connects routers input and output ports
- output ports : receives packets and transmits to outgoing links . handles link-layer and physical functions , performs scheduling and queuing .
- routing processor : performs control plane functions . In traditional it executes the routing protocols , for SDN it communicates with the remote controller in order to receive forwarding table entries . executed in softwares .

INPUT PORT PROCESSING & DESTINATION BASED FORWARDING



- line termination implements physical layer .
- data link processing implements link layer
- lookup , performs the forwarding table to look up to the output port . forwarding table is either computed & updated by routing processor or received from SDN remote controller .
- for destination address , router matches prefix of packets destination address with the entries in the table .
- if there are multiple matches , then the router uses longest prefix matching rule . and forwards it to respective link interface .

SWITCHING FABRICS

→ packet may be temporarily blocked from entering SF if packets from other input ports are currently using SF.

→ a blocked packet will be queued at input port and then scheduled.

→ switching via memory

- direct control of CPU

- Input port signaled the routing processor via an interrupt →

- packet copied to processor memory → routing processor extract

- dest address from header → output port lookup → copied packet to output ports buffer.

- no two packets can be forwarded at the same time.

→ switching via bus

- input port to output port over a shared bus without intervention of routing processor.

- input port appends a label and so all output ports see the label, but only the port whose ID matches, accepts it, removes label and sends to outgoing link.

- since only one packet can use a bus at a time, switching capacity is limited

→ switching via an interconnection network

- connects N ports to N output ports.

- N input buses (vertical), N output buses (horizontal)

- each intersection is a crosspoint that can be opened or closed

- multiple packets forwarded simultaneously

$A \rightarrow Y, B \rightarrow X$

- crossbar switch is non-blocking

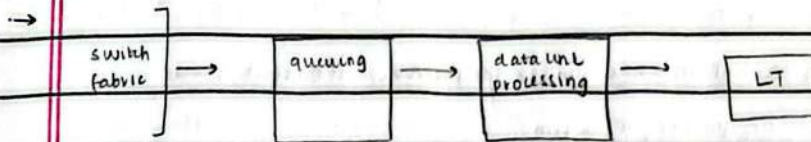
- if two inputs want the same output port, then one must wait.

- to allow multiple input to same output, use of multi-stage

switching fabric, has complex scheduling; Cisco CRS uses

3-stage non-blocking architecture.

OUTPUT PORT PROCESSING



- takes packet stored in output port's memory & transmits them to output link.

INPUT AND OUTPUT PORT QUEUING

→ input queuing

- packets arrive at input ports faster than the switching fabric can move them to output ports.
- packets wait in queue
- each input queue uses FCFS
- Head of line blocking: packet at the front of queue is blocked, it prevents the packets behind it from being transferred, even though they are destined for free output ports

→ output queuing

- multiple packets are delivered by switch fabric to same output port, faster than the output link can transmit them
- output port can transmit only one at a time, so remaining $N-1$ packets must queue and in that time another burst of packet can arrive.
- increases delay, packet loss and potential buffer overflow.
 - ↳ can send signal congestion early to senders.
 - ↳ mark packets before buffer is full.
 - ↳ drop arriving packet when buffer is full (drop tail)

old rule → $B = RTT \times C$

modern view → $B = RTT \times C \sqrt{N}$

$$\sqrt{N}$$

- too much buffering can increase delays. → higher RTT

PACKET SCHEDULING

→ first come first serve

- schedules packet in exact order they arrive at output queue.
- transmits one packet at a time.
- transmission time is fixed.
- no starvation
- large packets can delay small packets

→ priority scheduling

- always start serving from high priority.
- priority queue classifies into priority classes
- within same class, FIFO
- even if a high priority packet comes in between, then current packet completes because it is non-pre-emptive.

→ round robin

- packets classified into class
- prevents starvation
- never leaves link idle if any packet is waiting
- served alternately by class, not arrival time

→ weighted fair queuing

- each class is assigned a weight
- higher weight → more bandwidth

for class i bandwidth share = w_i

$$\sum w_j$$

if link rate is Throughput $i > R \times w_i$

$$\sum w_j$$

- this guarantee holds even in worst case, where all classes are backlogged.

THE INTERNET PROTOCOL

→ IPV4 datagram format

version	header length	type of service	datagram length (bytes)	
16-bit identifier			Flag	13-bit fragmentation
Time to live		upper-layer protocol	header checksum	
32-bit source IP address				
32-bit destination IP address				
options				
data				

IP ADDRESSING

- associated with an interface, rather than with the host or router containing that interface.
- each IP address is 32-bit long
- the boundary between host and physical link is called an interface; a router has multiple interfaces.
- subnet: simply a network that can physically reach each other.
IP addressing assigns an address to this subnet:

223.1.1.0/24, where /24 is known as subnet mask.

where leftmost 24 bits of the 32-bit quantity define subnet address.

- to determine the subnets, detach each interface from its host or router, creating islands of isolated network, with interfaces terminating the end points of the isolated network. Each of the isolated network is called a subnet.

→ classless Interdomain Routing (CIDR)

- modern IP address assignment & routing strategy.

- written as a.b.c.d/x

x = no. of bits in network prefix

- with CIDR, organizations are assigned a block of contiguous address ; they all share same prefix.
- routers outside of organization look at the prefix and use one forwarding table entry for entire block.
- reduces forwarding table size.

OBTAINING A HOST ADDRESS : DHCP

once an organization has obtained a block of address , it can assign individual IP address to the host and router.

DHCP is a client server protocol that automatically assigns IP configurations to host when they join a network.

DHCP also allows host to learn its subnet mask, default gateway and address of local DNS server.

DHCP often called as plug and play or zeroconf.

DHCP protocol is a four step process

• DHCP Discover :

- goal is to find a DHCP server,
- client sends a broadcast message of source IP : 0.0.0.0 and destination address 255.255.255.255

• DHCP offer :

- server responds with an address which is a broadcast because the client then can choose from amongst all servers present.

A each offer consists of proposed IP address, network mask, lease time (amount of time IP address will remain valid)

• DHCP request

- client accepts one offer and tells everyone the chosen IP address.

• DHCP Ack

- server sends acknowledgement and confirms IP address, lease duration and configuration parameters.
- client can renew its lease on an IP address.

OBTAINING A BLOCK OF ADDRESS

- ISP provides a block of address from a larger block.
- ISP owns a CIDR block which is subdivided into smaller contiguous blocks

example:

- ISP block → 200.23.16.0/20
- /20 means first 20 bits are network prefix, remaining 12 bits used to create sub-blocks.
- ISP splits /20 into 8 equal sized blocks.
 $120 \rightarrow 123 \rightarrow 3 \text{ bits borrowed} \rightarrow 2^3 = 8 \text{ subnets}$

- ICANN allocates IP address to Regional Internet Registries who then allocate it to ISPs
- it prevents IP address exhaustion, keeps global routing table scalable.

NETWORK ADDRESS TRANSLATION (NAT)

- many private devices share one public IP address.
- NAT router behaves to the outside world as a single device with a single IP address.
- NAT operation

- internal host sends packet

source IP: 10.0.0.1

source port: 3345

destination IP: 128.119.40.186

destination port: 80

- NAT router replaces source IP with its public IP & port as well

source IP: 138.76.29.7

source port: 5001

- NAT table entry created.

LAN: 10.0.0.1:3345

WAN: 138.76.29.7:5001

- server sends respond to

destination IP: 138.76.29.7

port: 5001

• NAT router looks up port 5001 and rewrites destination

to 10.0.0.1:3345 and forwards packet.

- port numbers allows NAT to distinguish
- NAT simplifies home network setup and works well with DHCP

IPv6

- motivation was that IPv4 address space was beginning to be used up
- by 2011, IANA allocated last IPv4 blocks.
- IPv6 designed to address shortage, simplify router processing.

version	traffic class	flow label	
payload length		next hop	hop limit
128-bit source address			
128-bit destination address			
Data			

- it uses 128-bit address for source & destination.
- 40-byte fixed header allows faster processing of IP datagram by a router.
- identifies types of flows.
- does not allow fragmentation & reassembly as it speeds up router IP forwarding.
- removed checksum as it was redundant; transport and link layer already perform this.
- no option field

TRANSITION FROM IPv4 TO IPv6

- 'flag day' upgrade not possible.
- internet already runs on IPv4
- solution: tunnelling
 - carry IPv6 packets inside IPv4 packets.

- tunneling
- IPv6 sender creates an IPv6 datagram
- datagram is encapsulated inside an IPv4 packet
- IPv4 routers forward it normally
- IPv6 receiver extracts the IPv6 packet and processes it.
 - IPv4 routers don't even know IPv6 is inside - they're blind carriers.
- why IPv6 adoption is slow?
- network layer changes are hard
- requires co-ordination across ISPs.

GENERALIZED FORWARDING

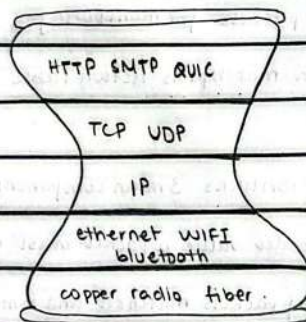
- traditional forwarding
 - limited to matching the dest IP address and forward packet
- generalized forwarding
 - packets are matched on many header fields & flexible actions are applied.
 - match can be based on link-layer fields, network layer fields etc.
 - actions include forward to one or more output, drop the packet, rewrite headers, mirror packets for monitoring.
 - each packet switch has a matchplus action table, called a flow table.
 - each flow table entry contains 3 main components:
 - match: specifies header value a packet must match.
 - counters: number of packets matched and time since last match.
 - actions: what to do when a packet matches.
 - openflow abstraction
 - router, switch, firewall, NAT

MIDDLEBOXES

- intermediary device on the path between a source and destination that performs functions other than standard IP forwarding.
- types of services
 - NAT translation: rewrite datagram header.
 - security: firewalls block traffic based on header-field values or redirect packets, DPI, IDS
 - caching, compression, load balancing.
- problems
 - high cost (capital + operational)
 - need for software stack, management tools
- solution
 - network function virtualization: implement middlebox functionality in software running on commodity hardware, instead of dedicated appliances.
 - example: virtual firewall, virtual NAT

IP HOURGLASS

- application → transport → network → link → physical



- IP is one universal protocol that all internet devices must implement.
- rise of middleboxes can be seen as slight widening of narrow waist

END TO END ARGUMENT

- internet places complexity in endpoints rather than network.
- reliable data transfer has functions implemented in network that causes packet loss due to crashes or failures.
- end to end intelligence ensures critical functions are handled where full knowledge exists.

NETWORK LAYER : CONTROL PLANE

ROUTING PROTOCOLS

- goal is to determine good path from sender to receiver. Typically it is the one with the least cost
- policy constraint may override pure cost minimization

GRAPH ABSTRACTION

- routing algorithms modeled using a graph

$$G = (N, E) \quad \therefore N = \text{set of routers}$$

E : set of edges

- each edge has a cost denoted as $c(x, y)$
- graph is undirected
- if all paths have equal cost - select the one with fewest hop.
- routing algorithms are either centralized or decentralized

• centralized algorithm

- computes the least cost path using complete, global knowledge about the network
- it knows all nodes, all links, all link costs.
- also called link state algorithm.

• decentralized algorithm

- computes least cost path is carried out in an iterative, distributed manner by the routers.
- each router knows only costs to its direct neighbour, exchange information
- also called distance vector algorithm.

- routing algorithms are either static or dynamic

• static

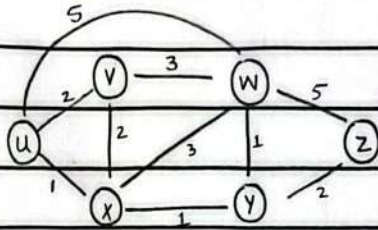
- routes change slowly, requires human intervention

• dynamic

- change paths as network traffic loads or topology change.

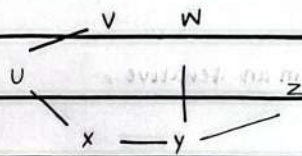
Dijkstra's Link State

- each router broadcasts link-state packets describing which neighbour has it and the cost
- each router independently runs the same algorithm.



step	N'	D(v), p(v)	D(w), p(w)	D(x), p(x)	D(y), p(y)	D(z), p(z)
0	u	2, u	5, u	1, x	∞	∞
1	ux	2, u	(1+3), x	(1+1), x	∞	∞
2	uxy	2, u	(1+1+1), y		(1+1+2), y	
3	uxyv		3, y			4, y
4	uxyvw					4, y

resulting least cost path :



forwarding table

v (u,v)

x (u,x)

y (u,y)

w (u,w)

z (u,z)

- number of nodes to search → $n(n+1)/2$

- worst case complexity → $O(n^2)$

- message complexity per router → $O(n)$

- link cost depend on current traffic load as routers repeatedly recompute shortest path, leads to routing oscillation.

INTRA-AS ROUTING : OSPF

- we assumed one network, where routers are identical and same routing algorithm causing 2 issues
- scale
 - storing routes to every destination is impossible.
 - distance vector convergence is slow.
 - administrative autonomy
 - internet = collection of ISPs.
 - each ISP want control over its own routing and privacy.

→ aggregate routers into regions known as 'autonomous systems'

- a group of routers under single administrative control.
- identified by a unique ASN
- routers inside an AS :
 - use same intra-AS routing protocol
 - share detailed routing information.
- there are two routing levels
 - Intra-AS : running within an AS → OSPF
 - Inter-AS : running between AS → BGP

OSPF ROUTING

- a link-state intra-AS routing protocol
- open standard (public)
- steps

- link state flooding
 - each router advertises its link, their cost, their up/down, status
 - advertisement goes to all routers.
- topology
 - every router builds a complete graph of AS.
- DIJKSTRA algorithm
 - each router computes a shortest path tree, with itself as a root.
 - produces forwarding table.

→ OSPF link weights

- set manually by network administrators
- all weights = 1 (min hop) or weight $\propto \frac{1}{\text{bandwidth}}$

→ OSPF runs directly over IP

→ must implement reliable delivery, flooding, database synchronization.

→ broadcast link state atleast after every 30 minutes.

→ features of OSPF

- Security
 - authentication prevents faked routers.
- equal cost multipath
 - for multiple short paths, OSPF can use all of them.
- unicast + multicast
 - uses same link state database.

→ OSPF hierarchy

- area : smaller link state
- area border routers : connect areas, summarize info.
- backbone area : connects all areas.
- boundary routers : connects to other AS.

ROUTING AMONG THE ISP : BGP

→ Inter-AS routing requires co-ordination

→ BGP is described as the protocol that glues the internet together.

→ communicating AS runs the same inter AS routing protocol

BORDER GATEWAY PROTOCOL (BGP) ROUTING

- each router has a forwarding table to decide the output link.
- BGP are not routed to specific destination address, instead uses CIDR prefixes
- routers forwarding tables have entries as (prefix, interface)
- BGP provides each router

- prefix reachability

- allows each subnet to advertise its existence

- without BGP subnets will be invisible, unreachable.

- best route selection

- a router may learn multiple routes to the same prefix

- BGP must compare them and select one best route

- BGP path advertisement

- if there are 3 AS and AS3 contains a subnet with prefix x, it

- means all IP address that start with prefix x belong to AS3

- each router of an AS falls in either gateway router or internal router

- gateway router located at the edge of an AS that connects to router in other AS's

- internal router connects only to host and routers inside same AS

- goal is to advertise reachability information for prefix x to all

routers

example: AS2 AS3 x

- types of BGP connection

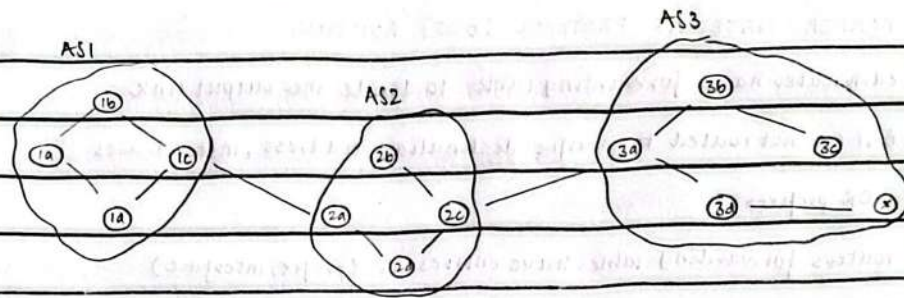
- External BGP (ebgp)

- exchange routing info across AS boundaries.

- Internal BGP (ibgp)

- used to distribute information inside an AS.

- every router connects to every router.



gateway router 3A sends → AS3 x via eBGP to gateway router 2C

gateway router 2C sends → AS3 x via iBGP to all routers in AS2

gateway router 2A sends → AS2 AS3 x via eBGP to gateway router 1C

gateway router 1C sends → AS2 AS3 x via iBGP to all routers in AS1

→ there can be multiple path so BGP must compare and make choice.

→ terminologies

- route : when a route advertises reachability it sends

prefix + attributes

- attributes : (1) AS-Path : list of AS numbers, shows the route

it has passed through. If a router

sees its own AS number, it rejects

the route, accepting it would create a

loop.

- (2) Next-hop : IP address of the router interface

that begins the AS-PATH.

→ hot potato routing

- the route chosen is the route with the least cost to NEXT-HOP router

- BGP does not look at full internet path

- cost is calculated through number of links.

- determine interface that leads to the gateway.

- update forwarding table.

→ route selection

• BGP sequentially invokes the following elimination rules until one route remains:

- local preference: a number assigned by AS, a higher number is more preferred
- shortest AS-path: from the remaining routes, shortest AS-path is selected - looks at number of hops
- hot potato routing: from remaining routes it compares route with same local preference and same AS-path length. - choose the route whose NEXT-HOP is closest, use OSPF to measure cost.

SDN CONTROL PLANE

→ Key characteristics

- flow-based forwarding
- packet forwarding based on any number of header field values and not just destination IP, or longest prefix matching.
- SDN control plane compute, manage and install flow table entries.
- separation of data plane & control plane.
 - data plane consists of network switches
 - they execute match + action
 - control plane consists of servers and softwares.
 - they decide what flow exist, how packets should move
- network control functions
 - control logic is outside, remote, centralized
 - software, executes on server that are both distinct and remote.
 - consists of SDN controller and network control applications.

- programmable network
- network becomes software defined
- software applications that decide path, block traffic, balance load.
- routing app that computes path e.g. Dijkstra, access control app.

SDN CONTROLLER AND SDN NETWORK CONTROL APP

- controller is organized into three layers.
- layer 1 : communication
 - controller is remote, controls switches, host & devices.
 - there must be a command to devices used as a protocol.
 - communication between controller and controlled devices cross the southbound interface.
 - southbound API = controller ↔ devices.
- layer 2 : network wide state management layer.
 - ultimate control decisions, up to date information.
 - controller maintains info about host, link, switches, flow tables
 - they need a global view
- layer 3 : interface to network-control applications
 - controller interacts with network-control applications through its northbound interface.
 - application do not talk to switches directly, they talk to controller.
 - allows to read/write network state.
- implemented on multiple servers to provide availability, fault tolerance.
- OpenFlow protocol
 - operates between SDN controller and SDN controlled switch.
 - it is the southbound protocol of SDN
 - defines how the controller talks to switches, how switches report back to the controller.
 - operates over TCP, on port 6653

- messages flowing from controller $\rightarrow$ controlled switches
 - configuration: controller to query, set switch configuration parameters, initialize switches
 - modify state message: add/delete/modify entries in switch's flow table.
 - read-state message: collect stats and counter values to monitor, load balancing.
 - send-packet message: controller can send packet out of specific switch port.
- message flowing from SDN controlled switch $\rightarrow$ controller
 - flow removed: flow table entry removed as resource constraints, timeout expired
 - port-status message: used by switch to inform change in port status.
 - packet in message: packet arrives at switch and does not match any flow table entry, so send to controller.

DATA AND CONTROL PLANE INTERACTION

- $\rightarrow$ as compared to traditional routing, in SDN, Dijkstra runs once, as network control applications outside the switches, switches send link-state updates only to SDN controller.
- $\rightarrow$ steps
 1. switch s_1 notifies the SDN controller using the OpenFlow port status message that link is down.
 2. SDN controller updates a link-state database
 3. The network-control application receives the notification.
 4. It then computes the new leastcost path through Dijkstra that runs once for the whole network.
 5. Routing app determines which switches are affected, what flow entries must change, Flowtable manager prepares updates
 6. OpenFlow protocol to update flow table entries.

- ORION provides intradomain services within google's network
- OpenDaylight and ONOS are modern SDN controllers.

INTERNET CONTROL MESSAGE PROTOCOL

- a network-layer control protocol used by routers and hosts to send information about IP delivery, not application data.
- it reports error
- above IP
- ICMP messages are carried inside IP datagrams.
- ICMP is IP's payload
- IP layer protocol field number = 1
- ICMP → IP → link layer.
- each ICMP contains type field (kind of message), code field, header (first 8 bytes of IP datagram)
- traceroute is a hack using ICMP and TTL
 - discovers all routers
 - round trip time to each one
- Working of traceroute
 - source sends IP datagrams that contains UDP segment & port number
 - controls TTL
 - at each router, TTL is reduced by 1 so TTL = 0, router drops the packet and sends back ICMP Type 11, code 0 (TTL expired)
 - when ICMP message arrives it has router name, router IP address and matches to a timer, computes round trip time.
 - stopping criteria
 - packet reaches destination, TTL is large
 - destination sends back type 3, code 3

NETWORK MANAGEMENT CONFIGURATION

→ network management

- monitor, test, configure, analyze, control devices so that performance and Quality of Service goals are met at reasonable cost.

→ components

• managing server

- central application that humans interact with
- collects data from devices, analyzes it, sends command to device

• managed device

- routers, switches, host, middleboxes
- each device has interfaces, routing protocols that can be configured.

• Data

- each managed device has data
- has configuration data (IP address), operational data (OSPF neighbour list, routing table), device statistics (counters and measurements)

• Network management Agent

- communicates with managing server, reads data and applies command.

• Network management protocol

- query device status, read data, write configuration

→ management approaches

- command line interface
 - operator logs into a device so direct control
- SNMP / MIB
 - SNMP used to query & set device data.
 - MIB is structured database of device variables.
- NETCONF / YANG
 - YANG a data modelling language
 - NETCONF is a protocol and supports transaction, rollback, multi-device changes.
 - network-wide, abstract

SNMP PROTOCOL

→ application layer protocol

→ only for monitoring and control

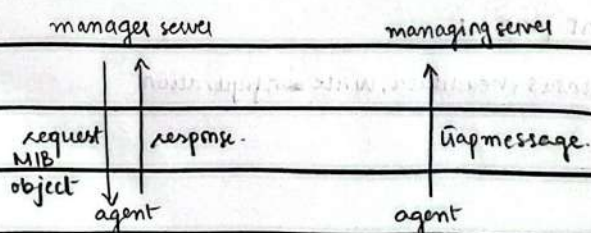
→ request-response mode

- managing server sends a request
- agent receives it, performs action, sends reply
- usually to read (query) / write (set) MIB object values.

→ trap messages

- sent by agent
- used to report interface up/down, authentication failure

→ MIB is a collection of managed objects.



NETCONF

- provides messaging to retrieve configuration data, modify, query operational data, receive notifications.
- uses remote procedural call (RPC)
- messages are XML formatted. exchanged over secure, connection oriented, reliable transport
 - <get> : get config and operational data.
 - <lock> <unlock> : prevent conflicting changes
- supports atomic multi-device transactions.
- can rollback on errors.
- key requirement in RFC 3535

YANG

- defines structure, syntax, semantics, define constraints.
- enables correctness checks, consistency, automated validation.
- for large scale.